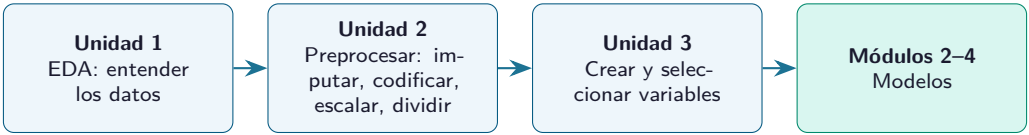
Repaso: preparación de datos

Unidades 1–3 · EDA, preprocesamiento, ingeniería y selección de variables

Mirar los datos · Preparar sin fuga · Crear y seleccionar variables

Sesión de repaso previa a la evaluación (≈30 min)

El hilo conductor: del dato crudo a un modelo confiable

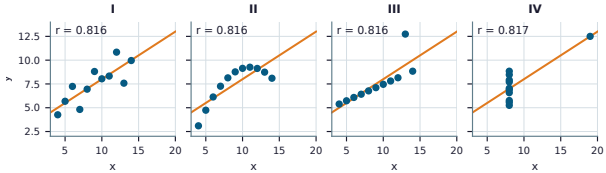


Idea central

Cada decisión de preparación debe **aprenderse solo del conjunto de entrenamiento** y vivir dentro de un `Pipeline`.

Sobre la evaluación

Notebook **individual** con tres partes (EDA · Preprocesamiento · Selección). Mezcla código y respuestas breves justificadas.



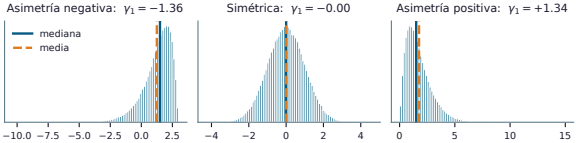
Cuatro conjuntos con **casi los mismos** resumen numérico (media, varianza, r) y formas muy distintas.

Lección

Los resúmenes **no sustituyen** a los gráficos: siempre hay que **mirar** los datos.

- **Centro:** media, mediana, moda
- **Dispersión:** varianza, desv. estándar, puntaje $z = \frac{x-\mu}{\sigma}$
- **Posición:** cuartiles y $\mathrm{IQR} = Q_3 - Q_1$
- **Forma:** asimetría y curtosis

Unidad 1 · Forma de la distribución y valores atípicos



- **Asimetría** > 0: cola derecha $\Rightarrow$ media $>$ mediana
- **Curtosis excedente** > 0: colas más pesadas que la normal
- Atípicos y colas: el **logaritmo** comprime la cola derecha

Regla del IQR (boxplot)

Atípico si

$$x < Q_1 - 1.5\,\mathrm{IQR} \quad \text{o} \quad x > Q_3 + 1.5\,\mathrm{IQR}$$

Marcar $\neq$ eliminar

Primero **investigar**: ¿error de captura o caso real? Opciones: corregir, transformar, conservar o modelar aparte. Justifica siempre la decisión.

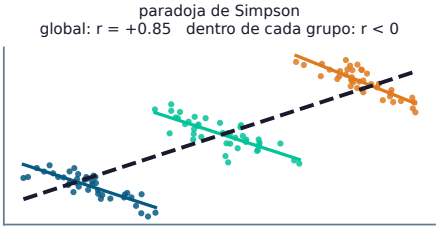
Pearson y Spearman

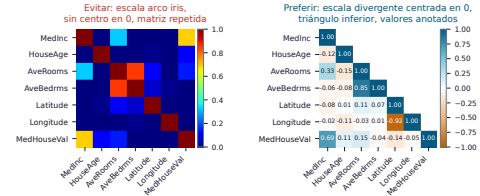
$$r = \frac{\mathrm{Cov}(X,Y)}{\sigma_X\,\sigma_Y} \in [-1,1]$$

- **Pearson:** asociación *lineal*; sensible a atípicos y escala
- **Spearman:** Pearson de los *rangos*; capta relaciones monótonas e **invariante** a transformaciones monótonas (log)
- $r = 0 \not\Rightarrow$ independencia

Cuidado

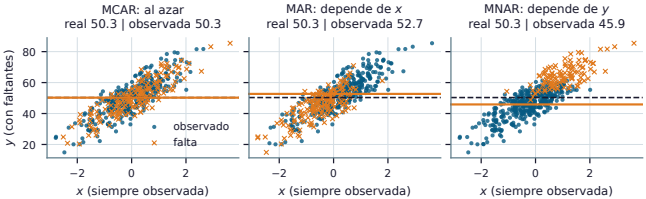
Correlación $\neq$ causalidad. Y mira los subgrupos: la **paradoja de Simpson** puede invertir el signo.





Matriz de correlación y mapa de calor: detecta **multicolinealidad** (variables explicativas muy correlacionadas $\Rightarrow$ coeficientes inestables).

Unidad 2 · Valores faltantes: ¿por qué faltan?



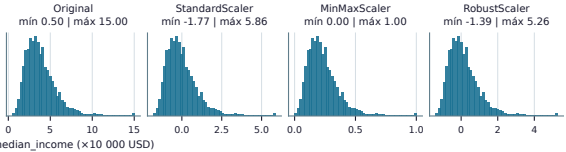
Tres mecanismos: el patrón de faltantes cambia lo que cada estrategia distorsiona.

MCAR	al azar, no depende de nada
MAR	depende de *otras variables observadas*
MNAR	depende del *propio valor* faltante

- **Cuantifica primero:** `df.isna().mean()` y por grupos
- **Borrar filas:** pierde datos y puede **sesgar** si no es MCAR
- **Imputar:** media/mediana/moda, KNN, iterativa; la mediana es robusta
- La imputación simple **encoge la varianza** y debilita correlaciones

Categóricas

- **Nominal** → `OneHotEncoder` (con `handle_unknown="ignore"` para categorías nuevas)
- **Ordinal** → `OrdinalEncoder` con el orden explícito
- Evita enteros en variables nominales: inventan un **orden y distancias** falsos
- Muchas o raras categorías: agrupar o codificar con cuidado



Escalar cambia la escala, **no la forma**.

`StandardScaler`	$(x - \bar{x})/s$; sensible a atípicos
`MinMaxScaler`	lleva a $[0, 1]$; muy sensible a atípicos
`RobustScaler`	mediana e IQR; robusto

¿Quién necesita escalar? kNN, SVM, regresión regularizada (Ridge/Lasso), descenso del gradiente. **Árboles: no**.

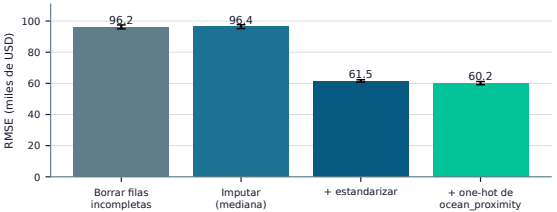
Unidad 2 · Dividir bien y evitar la fuga de datos

Entrenamiento / validación / prueba

- **Entrenamiento:** ajusta el modelo *y* el preprocesamiento
- **Validación (o CV):** elige modelo e hiperparámetros
- **Prueba:** se usa **una sola vez** al final
- Estratificar si hay clases o categorías raras

Regla de oro

`fit` solo con el entrenamiento; `transform` con todos. Medias, desviaciones, medianas y selecciones de variables son **parámetros aprendidos**.



Preprocesar antes de dividir da una evaluación **demasiado optimista**.

Unidad 2 · Pipeline + ColumnTransformer: la receta en código

```
pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("esc", StandardScaler())]),       num_cols),
    ("nom", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("oh",  OneHotEncoder(handle_unknown="ignore"))]), nom_cols),
    ("ord", OrdinalEncoder(categories=[["Bajo", "Medio", "Alto"]]),        ord_cols),
])
modelo = Pipeline([("pre", pre), ("reg", Ridge(alpha=1.0))])

rmse = -cross_val_score(modelo, X_train, y_train, cv=5,
                        scoring="neg_root_mean_squared_error").mean()    # sin fuga
modelo.fit(X_train, y_train)          # ajuste final; la prueba se usa una sola vez
```

- Cada pliegue de la CV **reajusta** imputador, escalador y modelo solo con su entrenamiento.
- Los hiperparámetros del preprocesamiento también se pueden buscar: `GridSearchCV(modelo, {"pre__num__imp__strategy": [...]})`.

- **Conocimiento del dominio:** razones y diferencias (p. ej., habitaciones por hogar, peso/talla^2)
- **Transformaciones:** $\log$, raíz, potencias para acercar a la simetría
- **Polinomios e interacciones:** `PolynomialFeatures`
- **Interacción:** el efecto de x_1 depende de x_2 ($x_1 x_2$)
- Un modelo **lineal** no capta relaciones en U: añade x^2

La explosión de columnas

Con p variables y grado d (sin término constante):

$$\binom{p+d}{d} - 1$$

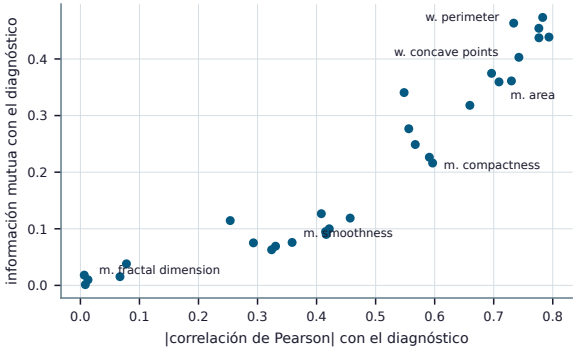
Ejemplo: $p = 5,\ d = 2 \Rightarrow \binom{7}{2} - 1 = 20$ columnas.

Riesgo

Más flexibilidad $\Rightarrow$ **sobreajuste**, más costo y menos interpretabilidad. Crea variables *dentro* del pipeline.

	Filtro	**Envoltura (RFE)**	**Integrado (Lasso)**
Idea	Puntúa cada variable por separado (sin modelo)	Entrena un modelo y elimina la peor, repitiendo	La penalización L_1 lleva coeficientes a **cero**
Ejemplos	varianza, correlación, `f_classif`, información mutua	`RFE`, `RFECV`	`SelectFromModel` con L_1
Costo	muy bajo	alto (muchos reentrenamientos)	bajo–medio
Ve redundancia	no	sí (conjunto)	sí (conjunto)
Límite	ignora interacciones; no ve el modelo	hereda las limitaciones del modelo	requiere escalar; elegir C (λ) por CV

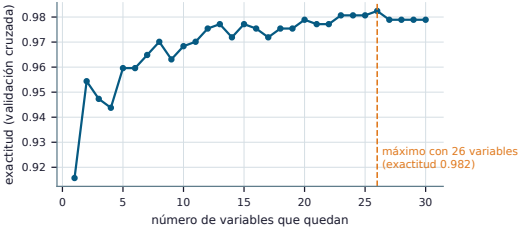
Unidad 3 · Filtros: qué ven y qué no ven



Información mutua frente a correlación: la MI detecta dependencias **no lineales**.

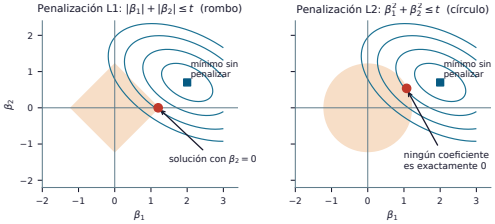
- **Umbral de varianza:** detecta columnas casi constantes, pero **depende de las unidades**. Una variable útil con escala pequeña se descarta; al estandarizar, todas tienen varianza 1.
- **Correlación / F:** captan relaciones lineales o diferencias de medias.
- **Información mutua:** cualquier dependencia (también en forma de U).
- **Redundancia:** dos copias casi idénticas puntúan alto ambas; elimina una o usa un método multivariado.

Unidad 3 · Envoltura e integrados: RFE y Lasso



`RFECV`: la validación cruzada elige **cuántas** variables conservar.

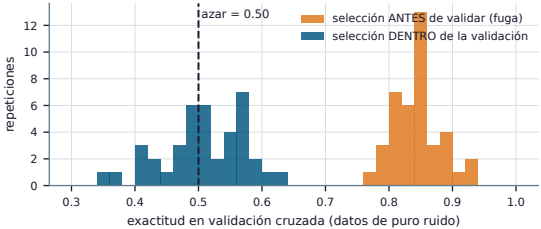
Estandariza antes de RFE para que los coeficientes sean comparables.



La geometría de L_1 (rombo) favorece soluciones con coeficientes **exactamente cero**.

C **pequeño** $\Rightarrow$ **penalización fuerte** $\Rightarrow$ **menos variables.**

Unidad 3 · La selección también va dentro de la validación cruzada



Con datos de **puro ruido**, seleccionar con todas las filas "descubre" patrones que no existen.

```python
# MAL: la seleccion ve y de todas las filas
Xsel = SelectKBest(f_classif, k=20).
    fit_transform(X, y)
cross_val_score(clf, Xsel, y, cv=5)

# BIEN: la seleccion se repite en cada pliegue
pipe = Pipeline([("sel", SelectKBest(f_classif,
     k=20)),
                 ("clf", LogisticRegression())
                    ])
cross_val_score(pipe, X, y, cv=5)
```

La regla es la misma de la Unidad 2: **todo lo que use y o estadísticos de los datos se ajusta solo con el entrenamiento**.

Autoevaluación rápida (piensa y luego avanza)

Unidad 1

1. Una variable tiene media $>$ mediana y curtosis excedente alta. ¿Cómo es su cola?
2. ¿Por qué Spearman no cambia al aplicar $\log$ a una variable positiva?

Unidad 2

3. Faltan datos solo en un grupo concreto. ¿MCAR, MAR o MNAR? ¿Qué riesgo hay al borrar filas?
4. ¿Qué fuga hay en `StandardScaler().fit_transform(X)` antes de `train_test_split`?

Unidad 3

5. ¿Por qué un filtro de F puede ignorar una variable en U que la MI sí elige?
6. ¿Cómo cambia el número de variables de Lasso al *bajar* C?

Autoevaluación rápida (piensa y luego avanza)

Unidad 1

1. Una variable tiene media $>$ mediana y curtosis excedente alta. ¿Cómo es su cola?
2. ¿Por qué Spearman no cambia al aplicar $\log$ a una variable positiva?

Unidad 2

3. Faltan datos solo en un grupo concreto. ¿MCAR, MAR o MNAR? ¿Qué riesgo hay al borrar filas?
4. ¿Qué fuga hay en `StandardScaler().fit_transform(X)` antes de `train_test_split`?

Unidad 3

5. ¿Por qué un filtro de F puede ignorar una variable en U que la MI sí elige?
6. ¿Cómo cambia el número de variables de Lasso al *bajar* C?

Respuestas breves

1. Cola derecha larga y pesada (valores extremos altos).
2. Spearman usa rangos; el $\log$ conserva el orden.
3. MAR (depende de otra variable observada). Borrar sesga la muestra contra ese grupo.
4. La media/desviación usan también las filas de prueba/validación $\Rightarrow$ evaluación optimista. Usar `Pipeline`.
5. F/correlación miden asociación lineal (diferencia de medias); la MI mide dependencia en general.
6. Disminuye: penalización más fuerte, más coeficientes en cero.

Errores frecuentes (y cómo evitarlos)

Error	**Lo correcto**
Eliminar atípicos "porque sí"	Investigar el origen; justificar la decisión
Imputar o escalar con *todos* los datos	`Pipeline`: `fit` con entrenamiento, `transform` con el resto
Codificar nominales con enteros	`OneHotEncoder(handle_unknown="ignore")`
Elegir variables viendo *todas* las etiquetas	Selección dentro del pipeline y de la CV
Umbral de varianza con unidades distintas	Verificar escalas; no usarlo como único criterio
Usar el conjunto de prueba varias veces	Prueba **una sola vez**, al final
Leer $r = 0$ como independencia	Graficar; usar Spearman o información mutua

Para el día de la evaluación

Practica con

- Las tres **actividades** de las unidades (notebooks y soluciones)
- Las **recetas** de cierre de cada presentación (6–7 pasos)
- Los gráficos: `hist`, `boxplot`, `scatter`, mapa de calor

Al entregar

- **No renombres** las variables que pide el enunciado
- Responde en la celda **"Respuesta:"** de cada pregunta
- **Reiniciar y ejecutar todo**: sin celdas con error
- Ejecuta la celda de **autoverificación**
- Sube solo el archivo `.ipynb`

Preguntas